Modeling of the experimental part

3.1 Goal of the experiments

We want to see if our modified PID controller keeps the water level close to the set point and moves the valve less than a normal PID, when the inflow and outflow are unknown. We also want to know what each extra part of the controller does.

3.2 The simulated process

The process is a tank with a valve. We use a simple model of the water balance, and it is the model of the class TankSimulator.

The variables are:

- PV(k): the water level at step k, in percent of the tank (from 0 to 100).
- SP(k): the set point, the level we want.
- u(k): the valve position, from 0 to 100.
- u0(k): the null position of the valve, where the inflow is equal to the outflow, so the level does not change. The controller does not know it. In S1 it is 50 all the time, and in other situations it can change.
- d(k): a disturbance added to the level, like rain (positive) or evaporation (negative). It is 0 in all the situations, except S4.
- r(k): the gain of the valve at step k, a random number.

We get the model from the water balance. The volume of water changes with the net flow, $A * \frac{dh}{dt} = Q_{in} - Q_{out}$, where A is the area of the tank and h is the level. The valve controls the inflow, $Q_{in} = c * u$, and the outflow Q_{out} is unknown. The null position is $u_0 = \frac{Q_{out}}{c}$, so the net flow is $c * (u - u_0)$. For a step of time Δt, the level changes with $(c * \frac{\Delta t}{A}) * (u - u_0)$. We call $r = c * \frac{\Delta t}{A}$ and we get the equation of the program. It is an integrator model, like the integrator models used for canal pools in [1] and [15].

$$PV(k+1) = PV(k) + r(k) * (u(k) - u_0(k)) + d(k)$$

The unknown inflow and outflow are inside u_0 and r. When they change, u_0 changes. What we know about the process:

- The level is constant only when $u(k) = u_0(k)$ and $d(k) = 0$. So the controller has to find u_0, and it does not know it.
- $r(k)$ is a whole number from 5 to 39, divided by 100, new at every step. It has 35 possible values with the same chance, so its mean is 0.22 and its standard deviation is about 0.10.
- In one step the level changes with maximum 0.39 · max(u_0, 100 − u_0), which is 19.5 for u_0= 50, plus the disturbance.

- The valve moves maximum 3% in one step: $|u(k) - u(k-1)| \leq 3$.
- If the new level would be outside 0 to 100, the program ignores that update and keeps the old level.
- The process runs on a timer, one step every 150 ms, and we count time in steps.
- The program starts with $PV(0) = 10,\ u(0) = 50$ and SP = 74.

We assume that the flow through the valve is linear around u_0, that $r(k)$ is independent from one step to the next, that the sensor reads the level exactly (we add noise only in situation S5), and that the controller makes one decision for each step of the process.

3.3 The controller

The controller makes one decision at every step. We write the steps in the order of the code (class ExtendedPid).

1. Error and dead-band: $e(k) = PV(k) - SP(k)$, in percent of the range (the range is 100). If e is positive, the level is too high. If $|e(k)|$ is smaller than the dead-band (0.2), the controller takes $e(k) = 0$, so it does not react to tiny errors.
2. Smoothing of the error: $e_s(k) = \frac{e_s(k-1) + e(k)}{2}$, with $e_s(0) = e(0)$.
3. Derivatives: $d_1(k) = e(k) - e_s(k-1)$and $d_2(k) = d_1(k) - d_{1s}(k-1)$, both rounded to 5 decimals. They are smoothed in the same way: $d_{1s}(k) = \frac{d_{1s}(k-1) + d_1(k)}{2}$ and $d_{2s}(k) = \frac{d_{2s}(k-1) + d_2(k)}{2}$. At the start $d_1(0) = d_2(0) = 0$ and $d_{1s}(-1) = d_{2s}(-1) = 0$. The steps are made by a timer at equal time intervals, so $dt = 1$ and we do not divide by time. The second derivative needs at least 3 values of the error, so in the first 5 steps the controller only collects data and the valve stays at 50.
4. Integral: $S(k) = S(k-1) + e(k)$, the right rectangle rule with $dt = 1$ and $S(-1) = 0$. S is limited to ±1500. If S > 0 and e < 0, or S < 0 and e > 0, we reset S to 0. For more precision we could use the trapezoid rule, $S(k) = S(k-1) + \frac{e(k) + e(k-1)}{2}$.The program has a setting for it, but we use the rectangle rule.
5. Derivative part: $D(k) = \frac{3 * d_{1s}(k) + 2 * d_{2s}(k)}{5}$. It uses $\frac{3}{5}$ of the rate and $\frac{2}{5}$ of the acceleration.
6. The three functions: $f(x) = sign(x) * \frac{ln(1 + k * |x|)}{k}$, with k = 0.1 for f_p and k = 0.01 for f_i. The third function is $f_d(x) = x$, without compression.
7. Output: the controller computes $v(k)$ and asks for the valve position $u^*(k)$, limited between 0 and 100. The valve goes there with a maximum 3% per step.

$$v(k) = -(K_p f_p(e_s(k)) + K_i f_i(S(k)) + K_d f_d(D(k))),\ u^*(k) = 50 + 0.5v(k)$$

The weights are Kp = 10, Ki = 0.2 and Kd = 25, and 50 is the bias of the controller. We found the weights with the tuning from 3.8.

Two more rules:

- Non-convergence: the error moves away from zero when (e > 0 and $d_{1s} > 0$) or (e < 0 and $d_{1s} < 0$). The controller counts the steps in a row like this. When it reaches 5 steps in a row, it cancels the derivative component: $d_{1s} = 0$, $d_{2s} = 0$ and $e_s = e$. This is done after the decision of that step, so it changes the next steps. Then the count starts again.
- Fallback: the fallback to the last convergent valve position is not in the simulation. It is useful only for a real process, where the null position of the valve can really change.

How the functions behave:

x	f(x) = x	f_p(x)	f_i(x)
1	1.00	0.95	1.00
5	5.00	4.05	4.88
10	10.00	6.93	9.53
50	50.00	17.92	40.55
100	100.00	23.98	69.31

Near zero, both functions are almost the line f(x) = x (the slope is 1). For big values they grow slowly, like a logarithm, so a big error or a big accumulated sum counts less than in the line. The constant k decides how strong this is. The function f_i uses a smaller k, because it is applied to the sum S, which is much bigger than the error. The sum can reach 1500, where $f_i(1500) = 277.3$. We chose this form of function, and we found the constants k together with the weights, in the tuning from 3.8.

With only the proportional part, the valve is $u^* = 50 - 5 * f_p(e_s)$, where 5 = 0.5 · 10. The level stops changing when $u^* = u_0$. For $u_0 = 62$ this gives $f_p(e_s) = -2.4$, so e_s is about −2.7 and the level stays 2.7 percent below the set point. A valve opening that is proportional to the error can leave the level blocked there. The integral keeps adding the error, so the valve keeps moving until the error is zero, and in this way the controller finds the null position u0 that it does not know. The derivative limits the movement if the level changes too fast.